WORKWISE • AI PROJECT INTELLIGENCE

Feature Implementation Guide

MongoDB Edition — Collections, Descriptions, Pipelines & Query Syntax

MongoDB version of the implementation guide. Every collection includes a plain-language description of its purpose, an explicit embedding-vs-referencing design note where relevant (this decision matters more in MongoDB than in a relational schema), a full field table, and suggested indexes. Every pipeline step uses real MongoDB query / aggregation syntax in place of SQL.

A	**Unified Data Model**	Every collection referenced by every feature below, defined once
01	**Overload Detector**	Reads: tasks, people, projects. Writes: suggestions
02	**Stuck Task Detector**	Reads: tasks, task_events, task_baselines. Writes: task_baselines, suggestions
03	**Conflict Detector**	Reads: tasks, projects, people. Writes: suggestions
04	**Velocity / Slippage Predictor**	Reads: project_snapshots, tasks. Writes: project_snapshots, suggestions
05	**Milestone Slippage Predictor**	Reads: milestones, tasks. Writes: suggestions
06	**Candidate Action Generator**	Reads: people, collaboration_history. Writes: into suggestions.candidate_actions
07	**Stage 4 Validation + Storage**	Reads: suggestions (in-memory). Writes: suggestions, validation_log
08	**Stage 5 Delivery + Feedback**	Reads: suggestions. Writes: suggestion_outcomes

A Unified Data Model

Every collection below is referenced by name in every feature section that follows.

TRIGGER	WRITES TO
N/A — reference section	N/A

MongoDB's document model changes a few schema decisions versus a relational design: some tables become embedded sub-documents, some composite primary keys become compound **`_id`** objects, and heterogeneous JSON payloads (like each suggestion's type-specific **`details`**) are a natural fit rather than something bolted on via a jsonb column. Each collection below calls out these decisions explicitly.

people

One document per user. This is the identity and capacity anchor that load-score, skill-matching, and collaboration-scoring all read from.

DESIGN NOTE skills is stored as a plain array so skill-overlap (Jaccard similarity) can be computed with MongoDB's array operators ($setIntersection, $setUnion) without a join to a separate skills collection.

Field	BSON Type	Description
`_id`	`ObjectId`	Person identifier
`name`	`String`	Display name
`team_id`	`ObjectId`	Team/project group membership
`skills`	`[String]`	Tag array, e.g. ['backend','security','react']
`capacity`	`Number`	Max sustainable load_score, default 5.0
`role`	`String`	e.g. admin, lead, member — used for delivery routing

SUGGESTED INDEXES

`{ team_id: 1 }` • `{ skills: 1 }`

tasks

Core work-item document. The single most frequently read and written collection in the system — almost every detection module starts here.

Field	BSON Type	Description
`_id`	`ObjectId`	Task identifier
`project_id`	`ObjectId`	Parent project reference
`assignee_id`	`ObjectId`	Currently assigned person (nullable)

type	String	e.g. bug, feature, review, audit — used for baseline lookup
priority	String	Low / Medium / High / Critical
status	String	Current workflow status
status_entered_at	Date	When current status began — drives dwell_days
due_date	Date	Task deadline
effort_estimate	Number	Story points or hours, nullable
reassignment_count	Number	Incremented on every assignee change
created_at	Date	Task creation time

SUGGESTED INDEXES
{ assignee_id: 1, status: 1 } • { project_id: 1 } • { type: 1, status: 1 }

task_events

Append-only raw event log — the Stage 0 foundation everything else is derived from. Every status change, reassignment, comment, and due-date edit is one document here, forever.

DESIGN NOTE Kept as its own collection rather than an embedded array inside tasks. A task's event history grows unbounded over its lifetime, and MongoDB documents cap at 16MB — embedding a growing log inside its parent is a well-known anti-pattern for exactly this reason. Reference by task_id instead.

Field	BSON Type	Description
_id	ObjectId	Event identifier
task_id	ObjectId	Task this event belongs to
event_type	String	status_change / reassignment / comment / due_date_edit
old_value	Mixed	Previous value, shape depends on event_type
new_value	Mixed	New value, shape depends on event_type
actor_id	ObjectId	Person who triggered the event
created_at	Date	Event timestamp

SUGGESTED INDEXES
{ task_id: 1, created_at: 1 } • { event_type: 1, created_at: -1 }

task_baselines

Small precomputed lookup collection, fully rebuilt every night. Read on every live Stuck Task check, so it stays tiny and cheap to query rather than recomputing statistics per-request.

DESIGN NOTE Uses a compound _id ({type, status}) instead of a separate auto-generated ObjectId — this collection has a natural composite key, and using it directly as _id gets you the uniqueness constraint and the primary lookup index for free, with no extra index needed.

Field	BSON Type	Description
`_id.type`	`String`	Task type — part of the compound primary key
`_id.status`	`String`	Workflow status — part of the compound primary key
`mu_dwell_days`	`Number`	Historical mean dwell time for this (type, status) pair
`sigma_dwell_days`	`Number`	Historical standard deviation
`n_samples`	`Number`	Count of historical tasks used — drives confidence
`last_computed_at`	`Date`	Last nightly refresh

SUGGESTED INDEXES

```
default _id index (compound key) — no additional index required
```

projects

One document per project. Deliberately thin — most project-level detail (velocity, burndown) lives in project_snapshots instead, keeping this collection stable and rarely-updated.

Field	BSON Type	Description
`_id`	`ObjectId`	Project identifier
`name`	`String`	Display name
`team_id`	`ObjectId`	Owning team
`deadline`	`Date`	Overall project deadline
`status`	`String`	active / completed / archived

SUGGESTED INDEXES

```
{ team_id: 1 }
```

project_snapshots

One document per (project, period) rollup — this is the hierarchical digest tier storage: weekly rollups feed monthly rollups, which feed the always-current project narrative.

DESIGN NOTE Kept separate from projects rather than as an embedded array for the same unbounded-growth reason as task_events: a multi-year project accumulates many snapshot periods, and this collection is queried independently (e.g. 'last 3 weekly snapshots') far more often than the full project document is needed alongside it.

Field	BSON Type	Description
`_id`	`ObjectId`	Snapshot document identifier
`project_id`	`ObjectId`	Project this snapshot covers
`period_type`	`String`	'weekly' or 'monthly'
`period_start`	`Date`	Start of window this rollup covers
`period_end`	`Date`	End of window this rollup covers
`velocity`	`Number`	Story points completed / period duration
`burndown_gap_days`	`Number`	See formula in Module 04
`summary_text`	`String`	AI-compressed narrative for this period
`created_at`	`Date`	When this snapshot was generated

SUGGESTED INDEXES

```
{ project_id: 1, period_type: 1, period_start: -1 }
```

milestones

One document per milestone. completed_at stays null while the milestone is open — this is the 'event' field the survival-analysis model treats as censored data until it's filled in.

Field	BSON Type	Description
`_id`	`ObjectId`	Milestone identifier
`project_id`	`ObjectId`	Parent project
`name`	`String`	Display name
`planned_due_date`	`Date`	Target completion date
`percent_complete`	`Number`	Current progress, 0-100
`owner_id`	`ObjectId`	Assigned owner, nullable
`completed_at`	`Date`	Null while in progress

SUGGESTED INDEXES

```
{ project_id: 1 } • { completed_at: 1 }
```

collaboration_history

Directional pairwise handoff stats between two people — feeds the Candidate Action Generator's collab_score.

DESIGN NOTE Compound _id ({person_a_id, person_b_id}) for the same reason as task_baselines: the natural key is the pair itself, so make it the primary key rather than adding a redundant ObjectId plus a separate unique index.

Field	BSON Type	Description
`_id.person_a_id`	`ObjectId`	Handoff source — part of compound primary key
`_id.person_b_id`	`ObjectId`	Handoff target — part of compound primary key
`total_handoffs`	`Number`	Count of tasks reassigned between these two people
`successful_handoffs`	`Number`	Of those, how many completed on time post-handoff

SUGGESTED INDEXES

`default _id index (compound key) — no additional index required`

suggestions

One document per generated suggestion — the shared envelope from the architecture spec, stored exactly as-is.

DESIGN NOTE This is where the document model is a genuinely better fit than relational: details is a heterogeneous, risk-category-specific shape (an Overload Risk details doc looks nothing like a Milestone Slippage details doc). In Postgres this needs a jsonb column with app-level validation; in MongoDB it's simply a normal nested sub-document, validated per risk_category via a $jsonSchema collection validator or an app-level schema library (e.g. Zod, Joi, Mongoose schemas).

Field	BSON Type	Description
`_id`	`UUID / ObjectId`	suggestion_id — matches the shared envelope schema
`risk_category`	`String`	Overload Risk / Stuck Task / Conflict / etc.
`risk_score`	`Number`	Stage 1/2 output, 0-100
`confidence`	`Number`	Stage 1/2 output, 0-1
`scope.type`	`String`	task / person / project / milestone
`scope.id`	`ObjectId`	The entity this suggestion is about
`details`	`Object`	Full Stage 2 structured payload, shape varies by risk_category
`candidate_actions`	`[Object]`	Ranked action array from Module 06
`phrased_text`	`String`	Stage 3 LLM output, or template fallback
`validated`	`Boolean`	Whether Stage 4's grounding check passed
`model_version`	`String`	Which rule/model version produced this

`created_at`	`Date`	Generation time

SUGGESTED INDEXES
`{ risk_category: 1, validated: 1, risk_score: -1 }` • `{ 'scope.id': 1 }` • `{ created_at: -1 }`

suggestion_outcomes

The single most valuable collection in the system — every accept/reject/outcome event a user generates, which is what eventually trains every future ML upgrade in Modules 04, 05, and 06.

Field	BSON Type	Description
`_id`	`ObjectId`	Outcome document identifier
`suggestion_id`	`ObjectId`	References suggestions._id
`shown_at`	`Date`	When delivered to a user
`action`	`String`	accepted / rejected / edited / ignored
`rejection_reason`	`String`	Quick-select chip value, nullable
`outcome_success`	`Boolean`	Did the real-world result confirm the suggestion was right
`outcome_recorded_at`	`Date`	When the outcome became knowable

SUGGESTED INDEXES
`{ suggestion_id: 1 }`

01 Overload Detector

Rule-based — no training data needed, runs directly against live tasks/people collections.

TRIGGER	WRITES TO
Event-driven: on any task status change, reassignment, or due-date edit + nightly full sweep as a safety net	suggestions (risk_category = 'Overload Risk')

Collection	Fields Read	Why
`people`	`_id, capacity, team_id`	Base capacity to compare load against
`tasks`	`assignee_id, priority, effort_estimate, due_date, status`	Build each person's active task list
`projects`	`team_id`	Scope the sweep to one team when event-driven

Pipeline

1 Get active people in scope

Aggregation joins tasks → projects to scope by team, then groups to distinct assignees.

```
db.tasks.aggregate([
  { $match: { status: { $nin: ['done','cancelled'] }, assignee_id: { $ne: null } } },
  { $lookup: { from: 'projects', localField: 'project_id',
               foreignField: '_id', as: 'proj' } },
  { $unwind: '$proj' },
  { $match: { 'proj.team_id': teamId } },
  { $group: { _id: '$assignee_id' } }
]);
```

2 Load each person's active tasks

Simple find with a projection — hits the { assignee_id:1, status:1 } index directly.

```
db.tasks.find(
  { assignee_id: personId, status: { $nin: ['done','cancelled'] } },
  { priority: 1, effort_estimate: 1, due_date: 1, status: 1 }
);
```

3 Compute weight per task, sum to load_score

In application code: for each task, urgency_w = 1 + max(0, (U - days_until_due)/U);
weight = priority_w × urgency_w × effort; load_score = Σ weight. (Full formula in the Algorithms doc.)

4 Check flag condition + compute confidence

If load_score > capacity_threshold AND critical_task_count ≥ min_critical: proceed to Step 5.
confidence = 0.95 × (tasks_with_effort_estimate / total_active_tasks).

5 **Write the suggestion document**

One document per flagged person, scope.type='person'. details follows the Overload Risk shape.

```
db.suggestions.insertOne({
  _id: suggestionId,
  risk_category: 'Overload Risk',
  risk_score: riskScore,
  confidence: confidence,
  scope: { type: 'person', id: personId },
  details: detailsDoc,
  candidate_actions: actionsArr,
  model_version: 'overload_v1',
  created_at: new Date()
});
```

6 **Downstream read**

Stage 3 (LLM phrasing) reads this document directly by _id.

Stage 5 (delivery) queries unread suggestions per team for the Team Load screen.

02 Stuck Task Detector

Two-part: an offline baseline rebuild, and a live per-task check against that baseline.

TRIGGER	WRITES TO
Baseline rebuild: nightly cron | Live check: on delay-worker tick	task_baselines (offline step) + suggestions (risk_category = 'Stuck Task')

Collection	Fields Read	Why
`tasks`	`type, status, status_entered_at`	Which (type, status) baseline applies, and current dwell
`task_events`	`event_type='status_change', created_at`	Historical dwell durations for the baseline
`task_baselines`	`mu_dwell_days, sigma_dwell_days, n_samples`	Precomputed baseline read at live-check time

Pipeline — Part 1: Offline baseline rebuild (nightly)

1 **Compute historical dwell time per completed status period**

$setWindowFields (MongoDB 5.0+) computes each event's next event time within the same task — the Mongo equivalent of a SQL window LEAD() function.

```
db.task_events.aggregate([
  { $match: { event_type: 'status_change' } },
  { $setWindowFields: {
      partitionBy: '$task_id',
      sortBy: { created_at: 1 },
      output: { next_change: { $shift: { output: '$created_at', by: 1 } } }
  }}
]);
// dwell_days = next_change - created_at, computed in application code
```

2 **Aggregate into mu/sigma per (type, status)**

$lookup joins back to tasks for type, then groups by (type, status) computing mean/stddev/count.

```
db.dwell_durations.aggregate([   // intermediate results from Step 1
  { $lookup: { from: 'tasks', localField: 'task_id',
               foreignField: '_id', as: 't' } },
  { $unwind: '$t' },
  { $group: {
      _id: { type: '$t.type', status: '$status' },
      mu: { $avg: '$days' },
      sigma: { $stdDevSamp: '$days' },
      n: { $sum: 1 }
  }}
]);
```

3 **Upsert into task_baselines**

upsert:true means insert-or-update in one call — matches the compound _id defined in Section A.

```
db.task_baselines.updateOne(
  { _id: { type: type, status: status } },
  { $set: { mu_dwell_days: mu, sigma_dwell_days: sigma,
            n_samples: n, last_computed_at: new Date() } },
  { upsert: true }
);
```

Pipeline — Part 2: Live per-task check

4 **Fetch the task + its baseline**

Two fast point-reads by _id — no join needed since task_baselines is keyed exactly by (type,status).

```
const task = db.tasks.findOne(
  { _id: taskId },
  { status_entered_at: 1, type: 1, status: 1 }
);
const baseline = db.task_baselines.findOne({
  _id: { type: task.type, status: task.status }
});
```

5 **Compute z-score, risk_score, confidence in application code**

dwell_days = now() - status_entered_at; z = (dwell_days - mu) / max(sigma, 0.5);
risk_score = 100 / (1 + e^-(z - z0)); confidence = min(1.0, n_samples / 30).
Flag only if z > z_threshold AND dwell_days > min_dwell_days.

6 **Write suggestion + re-queue the delay worker**

Same insertOne pattern as Module 01 Step 5, scope.type='task'; re-queue regardless of flag outcome.

```
db.suggestions.insertOne({ ..., scope: { type: 'task', id: taskId }, ... });
// re-enqueue: priorityQueue.push(taskId, runAt = now() + 1 day)
```

03 Cross-Project Conflict Detector

Pure aggregation query — no ML, no offline step. Entirely computed at query time.

TRIGGER	WRITES TO
Event-driven: on reassignment or due-date edit affecting a Critical-priority task	suggestions (risk_category = 'Cross-Project Conflict')

Collection	Fields Read	Why
`tasks`	`assignee_id, project_id, priority, due_date`	Find people with critical tasks spanning multiple projects
`projects`	`_id, name, deadline`	Project-level deadline fallback and display fields
`people`	`_id, name`	Display fields for the suggestion payload

Pipeline

1. **Find people with critical tasks on 2+ projects**
   $addToSet collects distinct project ids per assignee; $size + $match filters to 2 or more.

```
db.tasks.aggregate([
  { $match: { priority: 'Critical', status: { $nin: ['done','cancelled'] } } },
  { $group: { _id: '$assignee_id', projects: { $addToSet: '$project_id' } } },
  { $project: { project_count: { $size: '$projects' }, projects: 1 } },
  { $match: { project_count: { $gte: 2 } } }
]);
```

2. **For each candidate, pull their nearest critical deadline per project**
   $min inside $group is the Mongo equivalent of the SQL MIN() aggregate.

```
db.tasks.aggregate([
  { $match: { assignee_id: personId, priority: 'Critical',
              status: { $nin: ['done','cancelled'] } } },
  { $group: { _id: '$project_id', nearest_deadline: { $min: '$due_date' } } }
]);
```

3. **Score all project pairs in application code**
   For each pair (p_i, p_j) of that person's projects: overlap = max(0, 1 - |deadline_i - deadline_j| / W). conflict_score = Σ overlap across all pairs; risk_score = min(100, conflict_score × critical_mult × 100/N_max).

4. **Write suggestion**
   scope.type='person'. details.projects lists every project object involved.

```
db.suggestions.insertOne({ ..., scope: { type: 'person', id: personId },
                            details: detailsDoc, ... });
```

04 Velocity / Slippage Predictor

Deterministic features from project_snapshots, fed into a trained gradient boosting model.

TRIGGER	WRITES TO
Weekly, aligned with the snapshot-tier rollup cadence from the digest system	project_snapshots (features) + suggestions (risk_category = 'Velocity / Slippage')

Collection	Fields Read	Why
`project_snapshots`	`velocity, period_start, burndown_gap_days`	Last N weekly snapshots — the model's feature history
`tasks`	`effort_estimate, status`	Remaining scope calculation feeding burndown_gap_days

Pipeline

1. **Pull last N weekly snapshots for the project**
   Plain find + sort + limit — no aggregation needed, this collection is already shaped for this query.

```
db.project_snapshots.find(
  { project_id: projectId, period_type: 'weekly' },
  { period_start: 1, velocity: 1, burndown_gap_days: 1 }
).sort({ period_start: -1 }).limit(3);
```

2. **Compute velocity_trend via OLS in application code**
   Fit a simple linear regression of velocity against sprint index across the 3 rows from Step 1. Ephemeral per-request value — not written back to the DB.

3. **Assemble the full feature vector**
   Combine velocity_trend + burndown_gap_days (latest snapshot) + team_load_avg (live, reuses Module 01's load_score() across the project's team) + historical_completion_ratio (a per-team aggregate, computed once and cached).

4. **Call the trained model (3 quantile models: P10, P50, P90)**
   Model inference call, not a DB query — model artifacts load from your model registry (e.g. object storage + a model-serving process), independent of MongoDB.

5. **Compute confidence from interval width, run SHAP**
   confidence = 1 - clip((P90-P10)/interval_scale, 0, 1). If below threshold, stop — no suggestion write. Otherwise compute SHAP values, take top 3 by |impact| as top_feature_drivers.

6. **Write a new weekly snapshot AND a suggestion (if confidence passed)**
   Snapshot insert happens regardless of confidence — next week's trend calculation needs it either way.

```
db.project_snapshots.insertOne({
  project_id: projectId, period_type: 'weekly',
  period_start: start, period_end: end,
  velocity: velocity, burndown_gap_days: gap, created_at: new Date()
});

// only if confidence >= threshold:
db.suggestions.insertOne({ ..., scope: { type: 'project', id: projectId },
                           details: detailsWithShap, ... });
```

05 Milestone Slippage Predictor

Cox model trained offline on completed milestones; live inference reads only the current milestone.

TRIGGER	WRITES TO
Weekly, or on percent_complete update > 10 points since last check	suggestions (risk_category = 'Milestone Slippage')

Collection	Fields Read	Why
`milestones`	`percent_complete, planned_due_date, owner_id, completed_at`	Feature vector + the survival 'event'
`tasks`	`milestone_id, status`	dependency_count — open tasks still blocking this milestone

Pipeline — offline (model training, infrequent — monthly or on demand)

1 **Pull the full historical training set**

$lookup with a sub-pipeline counts open dependent tasks per milestone — the Mongo equivalent of a correlated subquery.

```
db.milestones.aggregate([
  { $lookup: {
      from: 'tasks', let: { mid: '$_id' },
      pipeline: [
        { $match: { $expr: {
            $and: [ { $eq: ['$milestone_id', '$$mid'] },
                    { $not: [ { $in: ['$status', ['done','cancelled']] } ] } ] } }},
        { $count: 'count' }
      ],
      as: 'open_tasks'
  }},
  { $addFields: { dependency_count:
      { $ifNull: [ { $arrayElemAt: ['$open_tasks.count', 0] }, 0 ] } } }
]);
```

2 **Fit the Cox model, compute C-index, persist model + C-index**

Model artifact + its C-index get versioned in the model registry, not in MongoDB.
CONFIDENCE for this model version = clip((C_index - 0.5) × 2, 0, 1), reused by every live prediction until the next retrain.

Pipeline — live inference

3 **Fetch current feature values for the milestone**

Point read by _id, filtered to still-open milestones.

```
db.milestones.findOne({ _id: milestoneId, completed_at: null });
```

4 **Run inference, compute predicted_delay_days**
predicted_completion = median survival time from S(t|x); predicted_delay = predicted_completion - planned_due_date.

5 **Write suggestion if predicted_delay exceeds threshold and model confidence passes**
scope.type='milestone'. owner field in details may be null — valid and expected per the architecture doc.

```
db.suggestions.insertOne({ ..., scope: { type: 'milestone', id: milestoneId },
                           details: detailsDoc, ... });
```

06 Candidate Action Generator

Not a standalone trigger — called inline by every module above, right before it writes its suggestion.

TRIGGER	WRITES TO
Inline: invoked synchronously from Steps 4-5 of Modules 01, 02, 03	Embeds directly into suggestions.candidate_actions (same insertOne as the calling module)

Collection	Fields Read	Why
`people`	`skills, capacity`	load_gap_score and skill_match_score inputs
`tasks`	`assignee_id, effort_estimate`	Current load per candidate target — reuses Module 01's load_score()
`collaboration_history`	`total_handoffs, successful_handoffs`	collab_score input

Pipeline

1. **Find eligible target people**
   Same team as the task, not the current assignee, projected to just the fields the scorer needs.

```
db.people.find(
  { team_id: teamId, _id: { $ne: currentAssigneeId } },
  { skills: 1, capacity: 1 }
);
```

2. **Compute each target's current load (calls Module 01's aggregation)**
   Reuses the exact load_score(p) logic from Module 01 Step 2-3 — call the same function so Overload Detector and Action Generator never disagree on someone's load.

3. **Pull collaboration history for each candidate pair**
   Point read on the compound _id — no query planner guesswork, this is a primary-key lookup.

```
db.collaboration_history.findOne({
  _id: { person_a_id: currentAssigneeId, person_b_id: candidateId }
});
```

4. **Score, rank, keep top-K in application code**
   action_score = w1·load_gap + w2·skill_match + w3·collab + w4·availability (weights in Algorithms doc §6).
   Sort descending, keep top 3 — becomes candidate_actions in the calling module's insertOne, no separate write.

07 Stage 4 — Validation + Storage

Runs entirely in application memory against the Stage 2 document already assembled — touches the DB only to persist the final result.

TRIGGER	WRITES TO
Synchronous, immediately after every Stage 3 LLM call	suggestions.phrased_text (update) + validation_log (on failure only)

Collection	Fields Read	Why
`suggestions`	`details, candidate_actions (already in memory from Stage 2)`	The grounded fact set — same document being finalized
`validation_log`	`suggestion_id, entity_ok, action_ok, format_ok`	Audit trail, written only when validation fails

Pipeline

1. **Run entity, action, and format checks in memory**
   No DB read required — the Stage 2 document is already the in-memory object that produced the LLM prompt.
   Full check logic is in the Algorithms doc §7.

2. **On pass: update the existing suggestion document**
   $set on the document already inserted in draft form by the calling module.

```
db.suggestions.updateOne(
  { _id: suggestionId },
  { $set: { phrased_text: llmOutput, validated: true } }
);
```

3. **On failure: log it, then update with the template fallback instead**
   The user still gets a correct suggestion (the template) — validation_log exists purely for debugging why the LLM path failed, not to block delivery.

```
db.validation_log.insertOne({
  suggestion_id: suggestionId, entity_ok: entityOk,
  action_ok: actionOk, format_ok: formatOk,
  raw_output: llmOutput, created_at: new Date()
});

db.suggestions.updateOne(
  { _id: suggestionId },
  { $set: { phrased_text: templateFallback, validated: false } }
);
```

08 Stage 5 — Delivery + Feedback Loop

Reads finalized suggestions for display, and writes back what the user did with each one.

TRIGGER	WRITES TO
Delivery: on suggestion write (real-time) or on screen load (pull) | Feedback: on user interaction	suggestion_outcomes (insert on show, update on user action)

Collection	Fields Read	Why
`suggestions`	`validated, phrased_text, risk_category, scope`	What to render on each product screen
`suggestion_outcomes`	`action, rejection_reason, outcome_success`	The single most valuable collection in the system

Pipeline

1 **Fetch suggestions for the current screen**

$lookup joins to people to scope by team, matching the delivery-surface mapping in the architecture doc §8.1.

```
db.suggestions.aggregate([
  { $match: { validated: true, risk_category: 'Overload Risk' } },
  { $lookup: { from: 'people', localField: 'scope.id',
               foreignField: '_id', as: 'person' } },
  { $unwind: '$person' },
  { $match: { 'person.team_id': teamId } },
  { $sort: { risk_score: -1 } }
]);
```

2 **Log the impression**

Written the moment a suggestion is actually rendered to a user, not just fetched by the API.

```
db.suggestion_outcomes.insertOne({
  suggestion_id: suggestionId, shown_at: new Date()
});
```

3 **Record the user's action**

Fired from the UI's accept/reject/edit/ignore handler.

```
db.suggestion_outcomes.updateOne(
  { suggestion_id: suggestionId },
  { $set: { action: action, rejection_reason: reason } }
);
```

4 **Record real-world outcome later (async, delayed)**

A separate scheduled job checks back on accepted suggestions after enough time has passed and writes the final verdict.

```
db.suggestion_outcomes.updateOne(
  { suggestion_id: suggestionId },
  { $set: { outcome_success: success, outcome_recorded_at: new Date() } }
);
```

5 **Feed back into Stage 0 / Stage 1**

This collection is the exact input to: (a) tuning confidence thresholds per Module 08 of the Algorithms doc, (b) training the learning-to-rank model in Module 06, and (c) retraining the Velocity and Milestone models.

Companion to: AI Suggestion Pipeline — Architecture Specification (v1.0) and Detection Module Algorithms (v1.0). All query/aggregation syntax above is illustrative MongoDB shell notation — adapt to your driver of choice (Node.js, PyMongo, etc.), but keep the read/write boundaries between modules exactly as shown, since that separation is what keeps each module independently testable.